IMDb Database — Data Dictionary

Course: Database Applications Development | Medina County Career Center
Database file: `imdb_class_2010plus.db`
Contents: ~19,000 movies and TV series from 2010 to present

What Is a Data Dictionary?

A **data dictionary** is a reference document that describes every table and column in a database. Think of it like the instruction manual for the database — before you write queries, you check here to know what data you're working with and how the tables connect.

Table of Contents

1. Titles — Movies & TV Shows
2. Ratings — Audience Scores
3. Cast & Crew Credits
4. Directors & Writers
5. People — Actors, Directors & More
6. Relationship Diagram

---

1. Titles — Movies & TV Shows (`title_basics`)

What it is: The main catalog of every movie and TV series in the database. One row = one title.
Rows: ~19,462
Nickname: "The Netflix browse page"

Friendly Name	Column Name	Data Type	Role	Description
Title ID	`tconst`	TEXT	🔑 **Primary Key**	Unique identifier for each title assigned by IMDb. Format: `tt` followed by numbers (e.g., `tt1375666` = *Inception*). Every other table uses this ID to refer back to a movie or show.
Type	`titleType`	TEXT		Whether this is a `movie` or `tvSeries`. Use in WHERE to filter one or the other.
Title	`primaryTitle`	TEXT		The release title you'd recognize — the name on the poster.
Original Title	`originalTitle`	TEXT		The title in the original language if different from English (e.g., foreign films).

Friendly Name	Column Name	Data Type	Role	Description
Adult Content Flag	`isAdult`	INTEGER		`0` = not adult content, `1` = adult content. Our database is pre-filtered to `0` only.
Release Year	`startYear`	INTEGER		The year the movie was released or the TV series began. Used heavily for filtering and sorting.
End Year	`endYear`	INTEGER		For TV series only — the year the show ended. `NULL` means still airing or unknown.
Runtime	`runtimeMinutes`	INTEGER		Length in minutes. May be `NULL` for titles without this info — always use `IS NOT NULL` if you sort by this.
Genre(s)	`genres`	TEXT		Comma-separated genre tags (e.g., `Action,Adventure,Sci-Fi`). Use `LIKE '%Genre%'` to filter.

Example row:

```
tconst     = tt1375666
titleType  = movie
primaryTitle = Inception
startYear  = 2010
genres     = Action,Adventure,Sci-Fi
runtimeMinutes = 148
```

---

2. Ratings — Audience Scores (`title_ratings`)

What it is: The IMDb crowd-sourced rating and vote count for every title. One row = one title's rating.
Rows: ~19,462 (one for every title in `title_basics`)
Nickname: "The star ratings you see on IMDb"

Friendly Name	Column Name	Data Type	Role	Description
Title ID	`tconst`	TEXT	🔗 **Foreign Key** → `title_basics.tconst`	The same unique title ID used in `title_basics`. This is how the two tables link — match on `b.tconst = r.tconst` in a JOIN.
Average Rating	`averageRating`	REAL		The IMDb user rating on a scale of **1.0 to 10.0**. This is a weighted average of all user votes.

Friendly Name	Column Name	Data Type	Role	Description
Vote Count	`numVotes`	INTEGER		How many users submitted a rating. More votes = more reliable score. Filter with `numVotes > 10000` to remove obscure titles.

Example row:

```
tconst        = tt1375666   ← same ID as Inception in title_basics
averageRating = 8.8
numVotes      = 2,767,665
```

---

3. Cast & Crew Credits (`title_principals`)

What it is: The bridge table linking people to the titles they worked on. One row = one person's credit on one title.
Rows: ~364,848
Nickname: "The movie credits roll"

Friendly Name	Column Name	Data Type	Role	Description
Title ID	`tconst`	TEXT	🔗 **Foreign Key** → `title_basics.tconst`	Which movie or TV show this credit belongs to.
Billing Order	`ordering`	INTEGER		The billing rank for this credit on this title. `1` = top-billed (the star), `2` = second billed, etc. Used in ORDER BY to sort cast lists correctly.
Person ID	`nconst`	TEXT	🔗 **Foreign Key** → `name_basics.nconst`	Which person this credit belongs to. Links to the People table.
Role Type	`category`	TEXT		What role this person had. Common values: `actor`, `actress`, `director`, `writer`, `producer`, `composer`. Use `IN ('actor','actress')` to catch both acting categories.
Specific Job	`job`	TEXT		A more specific job title within the category (e.g., `director of photography`). Often `NULL` for actors.

Friendly Name	Column Name	Data Type	Role	Description
Character(s) Played	`characters`	TEXT		For actors, the character name stored as JSON (e.g., `["Cobb"]`). May be `NULL` for non-acting roles.

Example rows:

```
tconst    = tt1375666  (Inception)
ordering  = 1
nconst    = nm0000138  (Leonardo DiCaprio)
category  = actor
characters = ["Cobb"]
```

---

4. Directors & Writers (`title_crew_person`)

What it is: A focused bridge table specifically for director and writer credits. Simpler than `title_principals` — only three columns.
Rows: ~116,970
Nickname: "The 'Directed by' and 'Written by' credits"

Friendly Name	Column Name	Data Type	Role	Description
Title ID	`tconst`	TEXT	🔗 **Foreign Key** → `title_basics.tconst`	Which movie or TV show this crew credit belongs to.
Person ID	`nconst`	TEXT	🔗 **Foreign Key** → `name_basics.nconst`	Which person is the director or writer.
Crew Role	`role`	TEXT		Either `director` or `writer`. No other values exist in this table.

Example row:

```
tconst = tt1375666  (Inception)
nconst = nm0634240  (Christopher Nolan)
role   = director
```

> **Note:** Directors also appear in `title_principals` with `category = 'director'`. Use `title_crew_person` when you only need director/writer info — it's simpler and faster.

---

5. People — Actors, Directors & More (`name_basics`)

What it is: The roster of every person in the database — actors, directors, writers, producers, and crew.
Rows: ~182,015
Nickname: "The actor/director profile page"

Friendly Name	Column Name	Data Type	Role	Description
Person ID	`nconst`	TEXT	🔑 **Primary Key**	Unique identifier for each person assigned by IMDb. Format: `nm` followed by numbers (e.g., `nm0000115` = Nicolas Cage).
Full Name	`primaryName`	TEXT		The person's name as it appears on IMDb. This is what you use in `WHERE n.primaryName = 'Name'` queries. Capitalization matters!
Birth Year	`birthYear`	INTEGER		The year they were born. May be `NULL` if unknown.
Death Year	`deathYear`	INTEGER		The year they died, if applicable. `NULL` for living people or unknown.
Main Profession(s)	`primaryProfession`	TEXT		Comma-separated list of what they're known for (e.g., `actor,producer,writer`).
Famous For	`knownForTitles`	TEXT		Comma-separated list of `tconst` values for the titles they're most associated with. Stored as IDs — you'd need to JOIN to see the actual titles.

Example row:

```
nconst            = nm0000115
primaryName       = Nicolas Cage
birthYear         = 1964
primaryProfession = actor,producer
knownForTitles    = tt0116367,tt0110413,tt0119217,tt0099685
```

Relationship Diagram

This diagram shows how all 5 tables connect. The **Primary Key (PK)** is the unique ID for that table. **Foreign Keys (FK)** are columns that point to a PK in another table — this is how JOINs work.

```
╔═══════════════════════════╗
║       name_basics         ║
║   (People Table)          ║
```

```
╚═══════════════════════════════╝
╔═══════════════════════════════╗
║  PK: nconst  (Person ID)      ║
║      primaryName              ║
║      birthYear                ║
║      deathYear                ║
║      primaryProfession        ║
║      knownForTitles           ║
╚═══════════════════════════════╝
                │
                │ nconst
                │ (Person ID)
        ┌───────┴───────┐
        │               │
        ▼               ▼

╔═══════════════════╗   ╔═══════════════════════╗
║ title_principals ║   ║  title_crew_person    ║
║ (Cast & Crew     ║   ║  (Directors &          ║
║  Credits)        ║   ║   Writers)             ║
╚═══════════════════╝   ╚═══════════════════════╝
╔═══════════════════╗   ╔═══════════════════════╗
║ FK: tconst ──────║───║──► title_basics        ║
║ FK: nconst ──────║───║──► name_basics         ║
║     ordering     ║   ║    role                ║
║     category     ║   ║    (director/writer)   ║
║     job          ║   ╚═══════════════════════╝
║     characters   ║             │ tconst
╚═══════════════════╝             │ (Title ID)
        │ tconst                  │
        │ (Title ID)              │
        └─────────────────────────┘
                  │
                  │
                  ▼

╔═══════════════════════════════════════════╗
║           title_basics                    ║
║        (Movies & TV Shows)                ║
╚═══════════════════════════════════════════╝
╔═══════════════════════════════════════════╗
║  PK: tconst        (Title ID)             ║
║      titleType     (movie/tvSeries)       ║
║      primaryTitle  (Name)                 ║
║      originalTitle                        ║
║      isAdult                              ║
║      startYear     (Release Year)         ║
║      endYear                              ║
║      runtimeMinutes                       ║
║      genres        (Comma-separated)      ║
╚═══════════════════════════════════════════╝
                    │
                    │ tconst
                    │ (Title ID)
                    ▼

╔═══════════════════════════╗
║       title_ratings       ║
║    (Audience Scores)      ║
╚═══════════════════════════╝
╔═══════════════════════════╗
║  FK: tconst  → title_basics ║
║      averageRating (1.0–10.0)║
```

```
╟      numVotes  (Vote Count)  ╢
╙────────────────────────────────╜
```

Key Lookup: Foreign Key Quick Reference

If you want to...	Start in...	JOIN to...	Match on...
Get a movie's rating	`title_basics`	`title_ratings`	`b.tconst = r.tconst`
Find who acted in a movie	`title_basics`	`title_principals`	`b.tconst = p.tconst`
Get an actor's name from a credit	`title_principals`	`name_basics`	`p.nconst = n.nconst`
Find all movies a person was in	`name_basics`	`title_principals`	`n.nconst = p.nconst`
Then get those movies' details	`title_principals`	`title_basics`	`p.tconst = b.tconst`
Find who directed a movie	`title_basics`	`title_crew_person`	`b.tconst = c.tconst`

Glossary

Term	What it means
Primary Key (PK)	The unique ID column for a table. Every row has a different value. No duplicates allowed.
Foreign Key (FK)	A column in one table that stores the Primary Key from another table. This is how tables are linked.
JOIN	The SQL command that glues two tables together by matching their FK and PK values.
NULL	A missing or unknown value. Not zero, not empty string — truly absent. Use `IS NULL` / `IS NOT NULL` to check for it.
Bridge Table	A table that exists mainly to connect two other tables (like `title_principals` connecting `name_basics` to `title_basics`).
Alias	A short nickname for a table in a query (e.g., `FROM title_basics b` — `b` is the alias). Saves typing.